

Chapter Overview

- 2.1 Error Handling in Visual Basic
- 2.2 Type of Errors
 - 12.2.1 Syntax Errors
 - 12.2.2 Logical Errors
 - 12.2.3 Run-Time Errors
 - 12.2.4 Common Run-Time Errors in Visual Basic
- 2.3 Handling Run-Time Errors in VB
- 2.4 Err Object
 - 12.4.1 Important Properties of Err Object
 - 12.4.2 Important Methods of Err Object
 - 12.4.3 Resuming Execution
 - 12.4.4 Raising an Error
- 2.5 Debugging Facilities in Visual Basic
 - 12.5.1 Immediate Window
 - 12.5.1.1 Displaying Values of Variables
 - 12.5.1.2 Using Debug Object
 - 12.5.1.3 Changing Values of Variables
 - 12.5.2 Locals Window
 - 12.5.3 Call Stack Window
 - 12.5.4 Watch Window
- 2.6 Breakpoints
- 2.7 Stepping Through Code

Short Questions**Multiple Choices****Fill in the Blanks****True/False**

12.1 Error Handling in Visual Basic

Errors may occur during execution even if the program is designed very carefully. Visual Basic provides facilities to handle such errors. Each error in Visual Basic has a unique number and description attached to it.

Visual Basic includes a default Error Handler that handles the situation when an error occurs. The error handler performs the following steps in case of an error:

- It displays the error number and a short description of the error.
- It terminates the application.

When an error occurs, the user cannot save any work or close the application properly. So it is very important to handle these errors by using error-handling code in procedures. A piece of code that handles run-time errors is also called **error handler**.

12.2 Type of Errors

The following types of errors can occur in VB programs:

- Syntax Errors
- Logical Errors
- Run-Time Errors

12.2.1 Syntax Errors

The set of rules and regulations for writing programs in a programming language is known as **syntax**. Syntax error is a type of error that occurs when the user writes an invalid statement in code window. For example, writing wrong spelling of a keyword is a syntax error. Visual Basic displays a message when a syntax error occurs. In the following figure, the keyword "to" is misspelled.

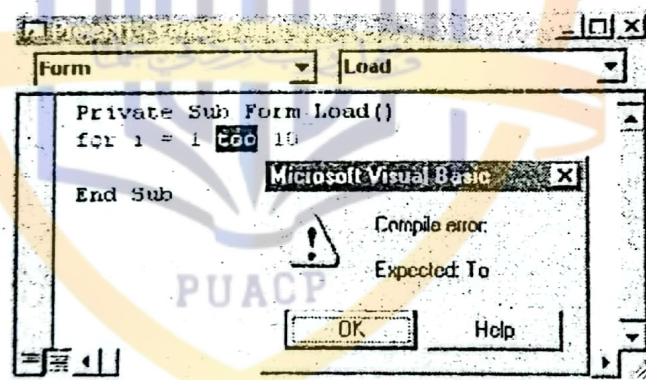


Figure: Syntax Error in Visual Basic

12.2.2 Logical Errors

Logical errors occur when the programmer writes such statements that are correct by syntax but wrong by logic. Such statements may produce unexpected results in the program. For example the programmer may have written a wrong formula to compute an answer. The formula will be executed without any error but will produce wrong results.

Logical errors are difficult to find because Visual Basic does not detect these errors. The programmer has to search the code and correct it.

12.2.3 Run-Time Errors

Run-Time errors occur during execution of the program. Such errors occur mostly due to invalid input by the user. For example the programmer has written a statement to divide two numbers given by the user. If the number is zero, the statement will become invalid. Visual Basic will display an error message and terminate the application.

Example

1. Start a new Standard EXE project.
2. Design the form as follows:

3. Add the following code in Click event of the button.

```

Project1 - Form1 (Code)
Click
Private Sub Command1_Click()
    Dim a As Integer, b As Integer, c As Integer
    a = Text1.Text
    b = Text2.Text
    c = a / b
    MsgBox "Result: " & c
End Sub

```

4. Run the project.
5. Enter 10 in first in textbox and 2 in second textbox and click Divide button. The result will appear as follows:

6. Now enter 10 in first in textbox and 0 in second textbox and click Divide button. This time an error will occur:

After displaying the above error message, Visual Basic closes the application. We can handle run-time errors by writing our own error-handling code.

12.2.4 Common Run-Time Errors in Visual Basic

Following are some common run-time errors in Visual Basic:

Error No.	Description	Error No.	Description
5	Invalid Procedure Call	53	File not found
6	Overflow	55	File already open
7	Out of memory	58	File already exists
9	Subscript out of range	76	Path not found
11	Divide by zero	423	Property or method not found
15	Type mismatch	482	Printer error

12.3 Handling Run-Time Errors in VB

Visual Basic does not provide centralized mechanism for handling errors. We need to write a separate error-handler for each procedure. There are three steps for creating errors handlers in Visual Basic:

- Enabling an Error Handler
- Writing code to handle a particular error
- Resuming the program execution

It means that when a run-time error occurs, an error handler is invoked, the code written in that error handler is executed and the program resumes its execution after handling the error. A label is used to create an error handler. A label is a name of a particular part of code window. The control is moved directly to an error handler when an error occurs during execution. The statement Go To is used to shift the control to an error handler.

12.4 Err Object

Err object contains information about run-time error of Visual Basic. It can be used to determine the number and description of the error. When an error occurs, the information of that particular error is stored in Err object.

12.4.1 Important Properties of Err Object

Some important properties of Err object are as follows:

Number

This property is used to get the unique number of run-time error.

Description

This property is used to get the description of run-time error.

Source

This property is used to get the name of application that generated the error.

12.4.2 Important Methods of Err Object

Some important methods of Err object are as follows:

Clear

This method clears all properties of the Err object after an error has been handled. This method is called automatically if we use the Resume statement, the On Error statement or when we exit a Sub or a Function procedure.

Raise

This method is used to generate an error. Generating an error is useful for testing and evaluation.

Example

The following example uses the same code used in the previous example. It includes an error handler in it. So if the user enters 0 in second textbox, the application does not halt.

1. Start a new Standard EXE project.
2. Design the form as follows:

3. Add the following code in Click event of the button.

```

Private Sub Command1_Click()
    On Error GoTo ErrorHandler
    Dim a As Integer, b As Integer, c As Integer
    a = Text1.Text
    b = Text2.Text
    c = a / b
    MsgBox "Result: " & c
    Exit Sub
ErrorHandler:
    MsgBox Err.Description
End Sub

```

Moving control to error handler

Error Handler

4. Run the project.
5. Now enter 10 in first in textbox and 0 in second textbox and click Divide button. This time an error will occur:

How It Works?

The first statement "On Error Go To ErrorHandler" tells Visual Basic to shift the control to the error handler if any error occurs in this procedure. Here, "ErrorHandler" is the name of error handler. It can be any valid name like "MyHandler" or "Ehander" etc. If any error occurs, the control shifts to the error handler and the message box is displayed. In this case,

the default error handler of Visual Basic does not work so the application is not suspended. The statement "Err.Description" uses a built-in object Err to display the description of error.

12.4.3 Resuming Execution

Error handler is used to handle the run time errors properly. The basic purpose is that the application should not be stopped unexpectedly. So when an error handler is executed, it should resume the application after handling the run time error. **Resume** statement is used to resume the execution of application properly. This statement can be used in different ways.

Statement	Description
Resume	Application resumes execution on the line of code that caused error.
Resume Next	Application resumes execution on the line of code following the line that caused the error.
Resume line	Execution resumes at the point designated by the line label or the line number.

Table: Resume statements

Example

Change the code in the previous example as follows and run it again.

```

Project1 - Form1 (Code)
Command1 Click
Private Sub Command1_Click()
    On Error GoTo ErrorHandler
    Dim a As Integer, b As Integer, c As Integer
    a = Text1.Text
    b = Text2.Text
    c = a / b
    MsgBox "Result: " & c
    Exit Sub

ErrorHandler:
    MsgBox Err.Description
    If b = 0 Then
        b = b + 1
        Text2.Text = b
    End If
    Resume
End Sub

```

Resume statement in the above code resumes the execution of application on the same line that causes the error.

12.4.4 Raising an Error

The following example demonstrates how can you raise an error.

Example

The following example gets a number from the user and displays its description.

1. Start a new Standard EXE project.

2. Design the form as follows:

The form titled "Raise Error" contains a text box with the prompt "Enter any error number to know its description" and a button labeled "Description".

3. Add the following code in Click event of the button.

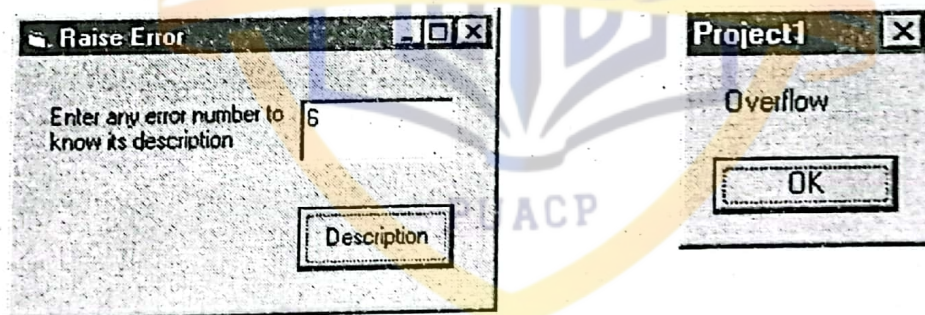
```

Project1 - Form1 [Code]
Command1 Click
Private Sub Command1_Click()
    On Error GoTo MyError
    Dim n As Integer
    n = Text1.Text
    Err.Raise n
    Exit Sub

MyError:
    MsgBox Err.Description
End Sub

```

4. Run the project.
5. Enter any number in textbox and click button. The result will appear as follows:



How It Works?

The above example gets a number from user and uses Raise method of Err object to raise an error of that number. The error handler displays the description related to the error number entered by the user.

12.5 Debugging Facilities in Visual Basic

Debugging is a process of removing errors from programs. Visual Basic also provides different facilities for debugging applications. Debug toolbar contains these facilities. The Debug toolbar can be displayed by selecting View > Toolbars > Debug.



Figure: Debug Toolbar

The user can find different errors easily by executing some portion of code. Visual Basic provides different debugging windows to determine the value of expressions and variables at different positions of your applications.

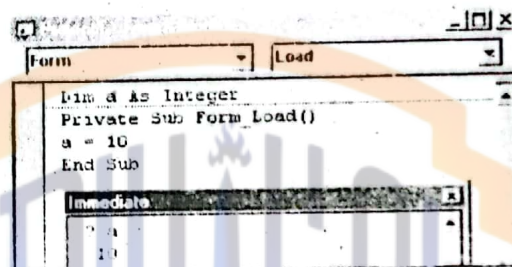
- Immediate Window
- Locals Window
- Call Stack Window
- Watch Window

12.5.1 Immediate / Debug Window

Immediate window is used to display the result of debugging statements. You can also write statements directly in this window. This window automatically opens in Break mode. Immediate window can be used for the following purposes:

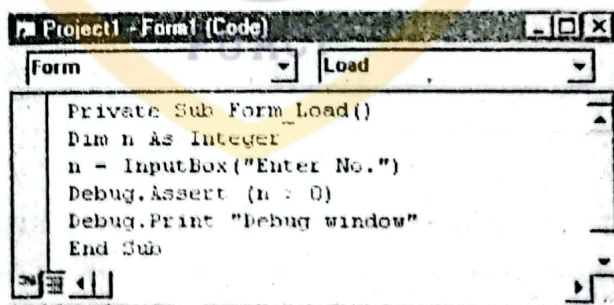
Displaying Values of Variables

You can execute any Visual Basic statement in **Immediate** window. You can display the values of any variable by writing Print statement directly in Immediate window. The question mark "?" is also used in place of Print statement.

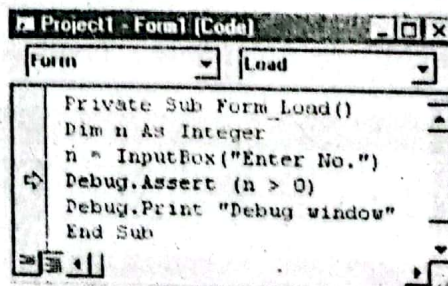


Using Debug Object

Any value can be displayed in Immediate window at run-time by using **Debug** object. This object has two methods i.e. Print and Assert. The Print method is used to display a message in Immediate window and Assert method is used to suspend the execution at the line on which the method is used.

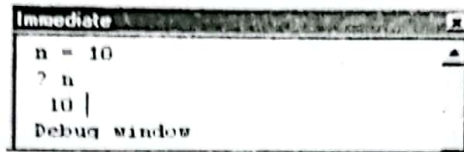


When you execute the above code, an input box will appear. If you enter a number greater than 0, the execution will continue and "Debug window" will be printed in Immediate window. Otherwise, the application will enter in Break mode as follows:



Changing Values of Variables

The values of a variable can also be changed in Immediate window. For example, the value of `n` can be changed when the above example is in Break mode due to Assert method.

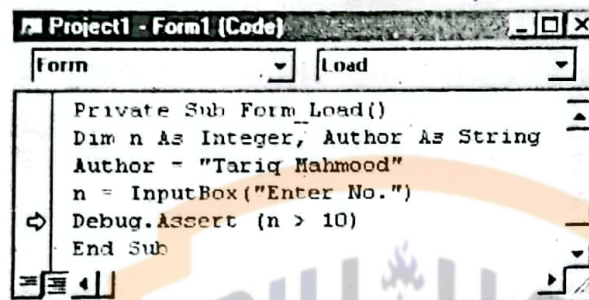


12.5.2 Locals Window

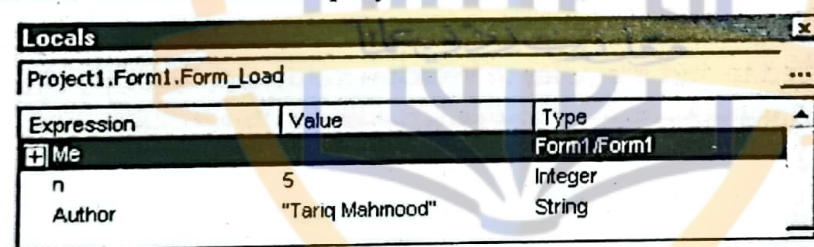
Locals window is used to display the values and types of all variables declared in the procedure where program was suspended. It cannot display the values of module variables.

Example

Write the following code in Click event of form:



When it is executed, an input box will appear. Enter 5 in it. The application will enter in Break mode and Locals window will display the following information:

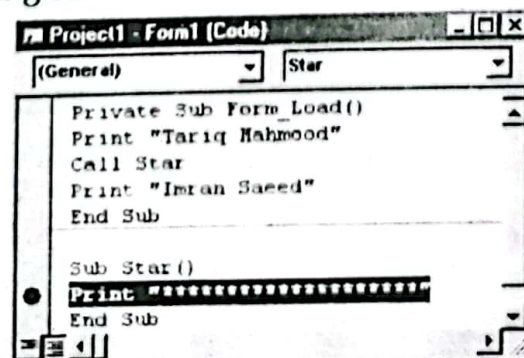


12.5.3 Call Stack Window

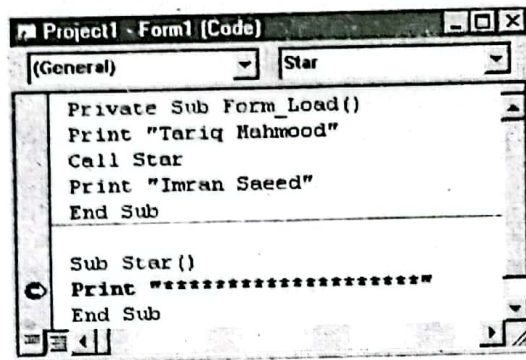
Call stack window is used to see the list of all procedures that were called to reach the current statement. You can see this window in Break mode by selecting Call Stack from Debug menu or by pressing CTRL+L.

Example

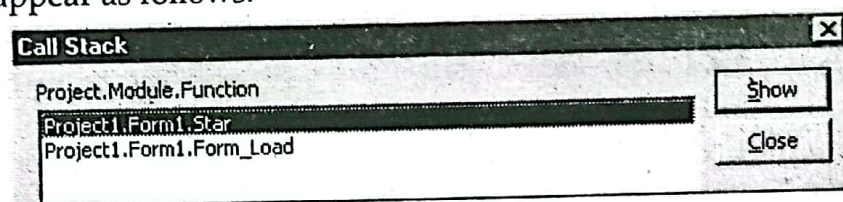
1. Start a new Standard EXE project.
2. Add the following code in Load event of the form.



3. Run the project. The form will enter in Break mode when the procedure is called.



4. Press CTRL+L or select Call Stack from Debug menu. The call stack window will appear as follows:



12.5.4 Watch Window

Watch window provides a way to monitor the value of variables. The values of local variables and module variables can be viewed by using Add Watch from debug menu or by dragging and dropping a variable into watch window.

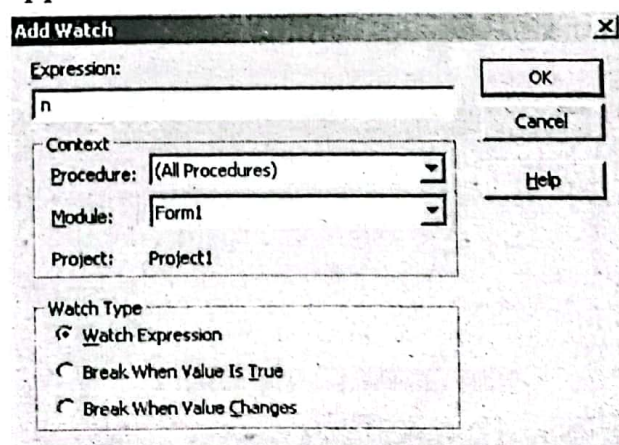
Watch Variables

The values of variables can be checked when a program pauses on a breakpoint. The following steps are used to add variables to Watch window:

- Right the variable name.
- Select Add Watch
- Click OK.

Example

1. Start a new Standard EXE project.
2. Add the following code in **General Declaration** section of the form.
Dim n as Integer
3. Right click n and select Add Watch from the popup window. The following window will appear:



4. Click OK. The variable will be added to Watch window as follows:

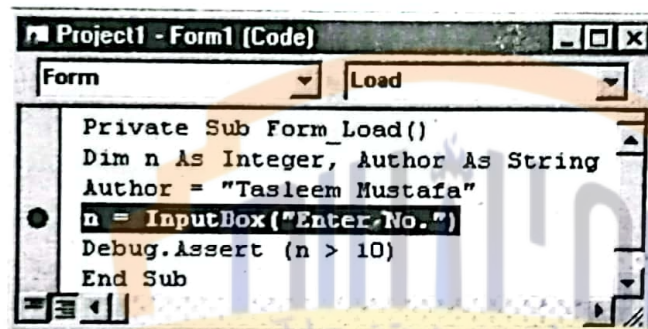
Watches			
Expression	Value	Type	Context
SS n	<Out of context>	Empty	Form1

12.6 Breakpoints

Breakpoint is a point in code where the program pauses. You can enter breakpoints to enter the application in Break mode for different purposes. For example you can check or change the values of different variables during application pause. Following steps are taken to insert a breakpoint in code:

- Place the cursor on the statement where you want to enter breakpoint.
- Click Toggle Breakpoint from Debug menu.

The statement will be highlighted and a filled circle will appear on the left side of the statement as follows:



In order to remove breakpoint:

- Place the cursor in the statement with breakpoint.
- Click Toggle Breakpoint from Debug menu.

12.7 Stepping Through Code

An easy way of finding an error in the application is to execute each statement one by one. Visual Basic provides the facility to execute statements individually. There are two ways to do so:

- Step Into
- Step Over

Step Into

Step Into is used to execute all statements one by one. If there is a procedure call, all statements in that procedure will also be executed one by one.

Step Over

Step Over is also used to execute all statements one by one. But if there is a procedure call, all statements of that procedure will be executed at once.

Short Questions

Q.1. What is a syntax error? When and why does it occur?

Syntax error is a type of error that occurs when the user writes an invalid statement in code window. For example, writing wrong spelling of a keyword is a syntax error. Visual Basic displays a message when a syntax error occurs. In the following figure, the keyword "to" is misspelled.

Q.2. What is a run-time error? When and why does it occur?

Run-Time errors occur during execution of the program. Such errors occur mostly due to invalid input by the user. For example the programmer has written a statement to divide two numbers given by the user. If the number is zero, Visual Basic will displays an error and terminate the application.

Q.3. What is a logic error? When and why does it occur?

Logical errors occur when the programmer writes such statements that are correct by syntax but wrong by logic. Such statements may produce unexpected results in the program. For example the programmer may have written a wrong formula to compute an answer. The formula will be executed without any error but will produce wrong results.

Q.4. What kind of error does Visual Basic find for you?

Visual Basic finds syntax errors for you.

Q.5. Which type of error is most difficult to find?

Logic errors are most difficult to find than other types of errors. It is difficult because Visual Basic does not find these errors. The programmer has to read the code carefully to find these errors.

Q.6. List the different steps in creating an error handler.

1. Enabling an error handler
2. Writing code that can handle possible errors
3. Continuing the execution of the program

Q.7. How does Visual Basic handle runtime errors if no error handler is implemented.

Visual Basic displays the error number with the error description and terminates the application.

Q.8. What is the role of a label in error handling?

The label identifies a location within the program where control transfers if an error occurs.

Q.9. Explain the function and use of the Err object.

The Err object stores information about an error that has occurred. The user can check its properties to determine error number and description of error. The name of the object or application that caused the error is stored in Source property. The Number property contains error number. The error is described in Description property.

Q.10. What is the difference between 'Resume Next' and 'Resume Line'? Explain.

In Resume Next, the application resumes processing on the line of code following the line that caused the error. In Resume line, execution resumes at the point designated by line label or line number. It must be in same procedure as error handler.

Q.11. What is breakpoint?

Breakpoint is a specific line on the execution is paused.

Q.12. What is displayed and what operations can be performed in Locals window, immediate window and watch window?

Locals window displays the values of all local variables in the current subprogram. These values may be modified by the programmer during debugging process. Immediate window is used to issue commands for immediate execution by Visual Basic. Watch window displays the values of all watch expressions. These values may be modified by programmer during debugging process.

Q.13. What happens when Visual Basic encounters breakpoint during program execution?

When Visual Basic reaches a breakpoint, Visual Basic pauses the program before the breakpoint line executes.

Q.14. Explain the difference between Step Into and Step Over.

Step Into or Step Over are used at design time. The program begins running and immediately transfers to break time. The project executes a single line at a time and displays the Code window with the current statement highlighted.

With Step Into, the next line of code is executed and the program pauses again in break time. If the line of code is a call to another procedure, the first line of other procedure will be displayed. Step Over command also executes one line of code at a time. The difference occurs when the code has calls to other procedures. Step Over does not display the code in the called procedure, but only displays the lines of code in the current procedure being analyzed.

Multiple Choice

1. A section of code designed to deal with runtime errors is called a(n):
a. Error object b. Runtime module c. Error handler d. Error function
2. Which type of errors give unintended results?
a. Runtime errors b. Syntax errors c. Logical errors d. Visual Basic errors
3. Errors that result from failing to follow the rules are:
a. Logical errors b. Syntax error c. Semantic error d. Exceptions
4. The statement that identifies the location of an error handler is:
a. On Error GoTo statement b. On Error Resume statement
c. Exit Sub statement d. Resume statement
5. Which of the tools is used to check the value of a variable?
a. Watch window, immediate window, and Locals window
b. Watch window, Show Next Statement, Locals window
c. Immediate window, Call Stack window, Locals window
d. Locals Window, Set Next Statement, Watch Window
6. Which of the following is NOT a debugging tool?
a. Copy b. Watch c. Breakpoint d. Step
7. Which tool is used to pause execution after a procedure while stepping through a program?
a. Step Into b. Step Over c. Run To Cursor d. Step Out
8. Which tool is used to change the value of a variable?
a. Immediate window b. Call stack window c. Step Over d. Run to Cursor
9. A program at runtime can be halted by:
a. Setting a breakpoint in code b. Setting a watch to break when the value changes
c. Both of the above d. None of the above
10. The output can be sent to Immediate window at runtime by using:
a. Print method b. Debug. Print c. Debug.Assert d. None
11. A good way to debug a program is to:
a. Set breakpoints near the code that causes an error
b. Execute a single line of code at a time
c. Watch variable values in a Watch Window
d. All of these are good debugging skill

Answers

1. c	2. c	3. b	4. a
5. a	6. a	7. b	8. a
9. c	10. b	11. d	12.

Fill in the Blanks

1. The Err object of Visual Basic is used for notifying _____.
2. _____ is enabled when we use On Error GoTo statement with the name of a label in the procedure.
3. To continue with the execution of procedure after an error is handled, Visual Basic provides the _____ statement.
4. Immediate window is also called _____.
5. _____ window is used to display the result of debugging statements.
6. _____ window is used to display the values and types of all variables declared in the procedure where the program was suspended.
7. _____ window is used to see the list of all procedures that were called to reach the current statement.

Answers

1. Error	2. Error handler	3. Resume
4. Debugging	5. Immediate	6. Locals
7. Call stack	8.	9.

True / False

1. Debugging is the process of removing errors from program.
2. Use Resume Next statement to instruct Visual Basic to resume processing at the statement immediately following the one that caused a runtime error.
3. Using 'Resume Next' will cause program execution to resume where run-time error occurred.
4. In Visual Basic, Err.Number will contain a description of the error encountered.
5. The Err object contains information about the syntax errors.
6. Print and Assert are methods of Debug object.
7. You can change the values of a variable using immediate window.
8. Breakpoint is a point in code where the program executed.
9. Clear and Raise are method of Err object.

Answers

1. T	2. T	3. F	4. F
5. F	6. T	7. T	8. F
9. T			